

# Week10\_Gueorgui\_Poklitar

August 1, 2026

\*\*\*\*\*Week 10 - K-Means Clustering\*\*\*\*\*

```
[1]: %pip install -q pandas numpy matplotlib scikit-learn kagglehub
```

[notice] A new release of pip is available: 26.0.1 -> 26.2

[notice] To update, run:

```
pip install --upgrade pip
```

Note: you may need to restart the kernel to use updated packages.

```
[2]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import sklearn
import kagglehub

from sklearn.cluster import KMeans
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (
    adjusted_rand_score,
    average_precision_score,
    calinski_harabasz_score,
    davies_bouldin_score,
    roc_auc_score,
    silhouette_samples,
    silhouette_score
)
from sklearn.metrics.pairwise import pairwise_distances
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import (
    MinMaxScaler,
```

```

    OneHotEncoder,
    RobustScaler,
    StandardScaler,
    normalize
)
from sklearn.utils.class_weight import compute_sample_weight

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 180)

print(f"scikit-learn version: {sklearn.__version__}")
print("All imports successful.")

```

scikit-learn version: 1.9.0  
All imports successful.

## Data Loading and Cleaning

```

[3]: path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

df = df.drop_duplicates().copy()

df["person_emp_length"] = df["person_emp_length"].fillna(
    df["person_emp_length"].median()
)

df["loan_int_rate"] = df["loan_int_rate"].fillna(
    df.groupby("loan_grade")["loan_int_rate"].transform("median")
)

df = df[df["person_emp_length"] <= df["person_age"]]
df = df[df["person_age"] <= 100]
df = df[df["person_emp_length"] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate: {df['loan_status'].mean() * 100:.2f}%")
print("\nMissing values:")
print(df.isna().sum())

```

Raw shape: (32581, 12)  
Clean shape: (32409, 12)  
Default rate: 21.87%

Missing values:

|               |   |
|---------------|---|
| person_age    | 0 |
| person_income | 0 |

```

person_home_ownership      0
person_emp_length          0
loan_intent                 0
loan_grade                 0
loan_amnt                  0
loan_int_rate              0
loan_status                0
loan_percent_income        0
cb_person_default_on_file  0
cb_person_cred_hist_length 0
dtype: int64

```

### Unsupervised Feature Preparation

```

[4]: numeric_features = [
    "person_income",
    "person_age",
    "person_emp_length",
    "loan_amnt",
    "loan_percent_income",
    "loan_int_rate"
]

categorical_features = [
    "loan_intent",
    "loan_grade",
    "person_home_ownership",
    "cb_person_default_on_file"
]

X = df[numeric_features + categorical_features].copy()
y_holdout = df["loan_status"].copy()

complete_mask = X.notna().all(axis=1)
X = X.loc[complete_mask]
y_holdout = y_holdout.loc[complete_mask]
df_model = df.loc[complete_mask].copy()

scaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features

```

```

    )
]
)

unscaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

numeric_only_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features)
    ]
)

X_scaled_full = scaled_preprocessor.fit_transform(X)
X_unscaled_full = unscaled_preprocessor.fit_transform(X)
X_numeric_only = numeric_only_preprocessor.fit_transform(X)

categorical_names = (
    scaled_preprocessor.named_transformers_["cat"]
    .get_feature_names_out(categorical_features)
    .tolist()
)
full_feature_names = numeric_features + categorical_names

X_scaled_full = np.asarray(X_scaled_full, dtype=np.float64)
X_unscaled_full = np.asarray(X_unscaled_full, dtype=np.float64)
X_numeric_only = np.asarray(X_numeric_only, dtype=np.float64)

print(f"Borrowers available for clustering: {X_scaled_full.shape[0]:,}")
print(f"Full representation dimensions: {X_scaled_full.shape[1]}")
print(f"Numeric-only representation dimensions: {X_numeric_only.shape[1]}")
print(f"Withheld default rate: {y_holdout.mean() * 100:.2f}%")
print(
    "\nThe label above is reported for reference only and is not used "
    "in any fit call in this notebook."
)

```

Borrowers available for clustering: 32,409  
Full representation dimensions: 25  
Numeric-only representation dimensions: 6  
Withheld default rate: 21.87%

The label above is reported for reference only and is not used in any fit call in this notebook.

### Feature Scaling: Why K-Means Cannot Be Run on Raw Columns

```
[5]: scale_summary = pd.DataFrame({
    "Feature": numeric_features,
    "Standard Deviation": [X[column].std() for column in numeric_features]
}).sort_values("Standard Deviation", ascending=False)

print("Raw Numeric Feature Scales")
print(scale_summary.round(4).to_string(index=False))

variance_ratio = (
    scale_summary["Standard Deviation"].max() ** 2
    / scale_summary["Standard Deviation"].min() ** 2
)

print(
    "\nRatio of largest to smallest *squared* standard deviation:",
    f"{variance_ratio:,.0f} to 1"
)

scaling_test_k = 4

scaling_comparison = []

for label, matrix in [
    ("Unscaled", X_unscaled_full),
    ("Standardised", X_scaled_full)
]:
    model = KMeans(
        n_clusters=scaling_test_k,
        n_init=10,
        random_state=42
    )
    labels = model.fit_predict(matrix)

    silhouette = silhouette_score(
        matrix,
        labels,
        sample_size=5000,
        random_state=42
```

```

)

income_by_cluster = pd.Series(
    df_model["person_income"].values
).groupby(labels).mean()

income_spread = (
    income_by_cluster.max()
    - income_by_cluster.min()
)

percent_income_by_cluster = pd.Series(
    df_model["loan_percent_income"].values
).groupby(labels).mean()

percent_income_spread = (
    percent_income_by_cluster.max()
    - percent_income_by_cluster.min()
)

scaling_comparison.append({
    "Preprocessing": label,
    "Silhouette": silhouette,
    "Calinski-Harabasz": calinski_harabasz_score(matrix, labels),
    "Income Spread Across Clusters": income_spread,
    "Debt Burden Spread Across Clusters": percent_income_spread,
    "Smallest Cluster Share": (
        pd.Series(labels).value_counts(normalize=True).min()
    )
})

scaling_comparison_df = pd.DataFrame(scaling_comparison)

print(f"\nK-Means at k={scaling_test_k}: Unscaled Versus Standardised")
print(scaling_comparison_df.round(4).to_string(index=False))

print(
    "\n A high silhouette on unscaled data is not evidence of a "
    "good segmentation. It reflects that one dominant column produces "
    "cleanly separated bands. The debt-burden spread column shows whether "
    "the partition distinguishes borrowers on anything beyond income."
)

```

#### Raw Numeric Feature Scales

| Feature       | Standard Deviation |
|---------------|--------------------|
| person_income | 52517.8700         |
| loan_amnt     | 6320.8851          |
| person_age    | 6.2104             |

|                     |        |
|---------------------|--------|
| person_emp_length   | 3.9838 |
| loan_int_rate       | 3.2135 |
| loan_percent_income | 0.1068 |

Ratio of largest to smallest \*squared\* standard deviation: 241,876,641,369 to 1

K-Means at k=4: Unscaled Versus Standardised

| Preprocessing | Silhouette | Calinski-Harabasz | Income Spread Across Clusters | Debt Burden Spread Across Clusters | Smallest Cluster Share |
|---------------|------------|-------------------|-------------------------------|------------------------------------|------------------------|
| Unscaled      | 0.5535     | 37730.1454        |                               |                                    | 805719.4592            |
| 0.1747        |            | 0.0015            |                               |                                    |                        |
| Standardised  | 0.1351     | 4370.0550         |                               |                                    | 60927.0771             |
| 0.1812        |            | 0.1207            |                               |                                    |                        |

A high silhouette on unscaled data is not evidence of a good segmentation. It reflects that one dominant column produces cleanly separated bands. The debt-burden spread column shows whether the partition distinguishes borrowers on anything beyond income.

## The Elbow Method

```
[6]: candidate_k_values = list(range(2, 11))

elbow_results = []

for k in candidate_k_values:
    model = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    )
    model.fit(X_scaled_full)

    elbow_results.append({
        "Clusters": k,
        "Inertia": model.inertia_
    })

elbow_df = pd.DataFrame(elbow_results)

elbow_df["Inertia Reduction"] = (
    elbow_df["Inertia"].shift(1)
    - elbow_df["Inertia"]
)

elbow_df["Percent Reduction"] = (
    100
```

```

    * elbow_df["Inertia Reduction"]
    / elbow_df["Inertia"].shift(1)
)

print("Elbow Analysis")
print(elbow_df.round(3).to_string(index=False))

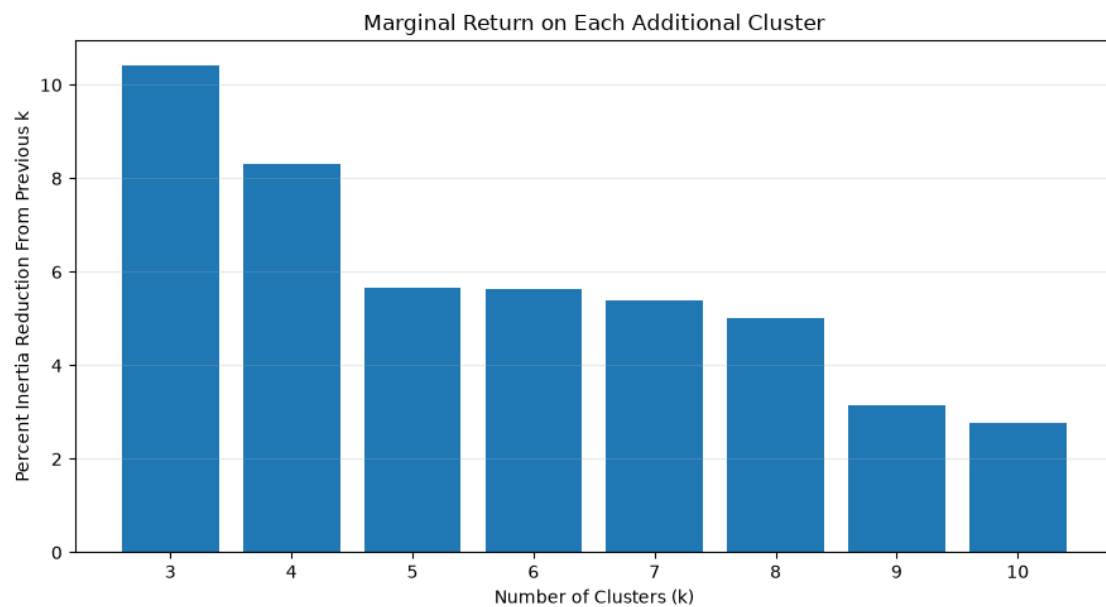
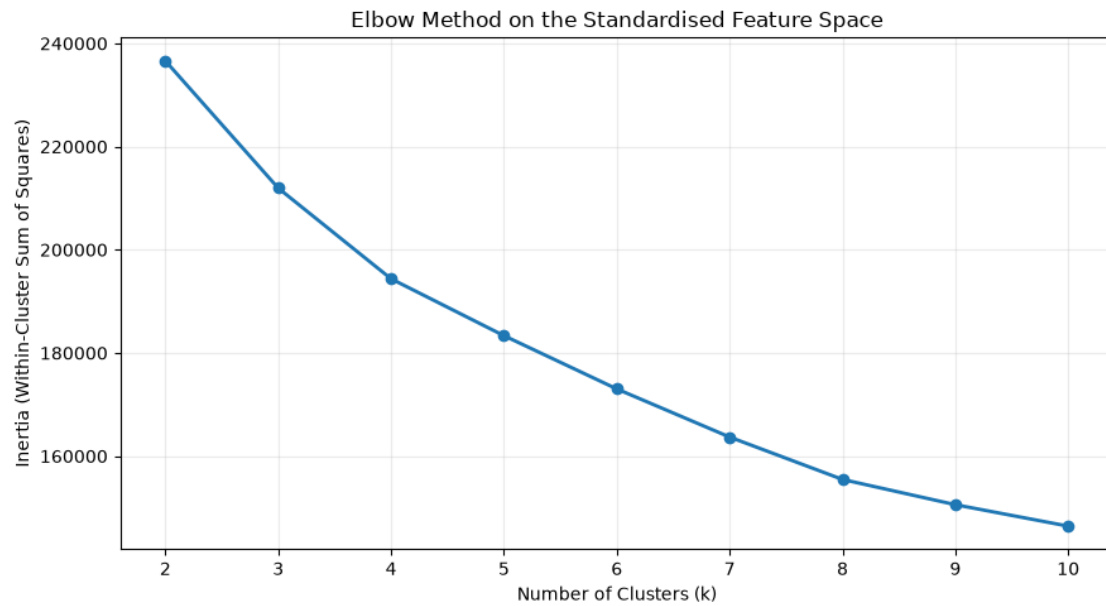
plt.figure(figsize=(9, 5))
plt.plot(
    elbow_df["Clusters"],
    elbow_df["Inertia"],
    marker="o",
    linewidth=2
)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia (Within-Cluster Sum of Squares)")
plt.title("Elbow Method on the Standardised Feature Space")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 5))
plt.bar(
    elbow_df["Clusters"],
    elbow_df["Percent Reduction"]
)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Percent Inertia Reduction From Previous k")
plt.title("Marginal Return on Each Additional Cluster")
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()

```

#### Elbow Analysis

| Clusters | Inertia    | Inertia Reduction | Percent Reduction |
|----------|------------|-------------------|-------------------|
| 2        | 236620.770 | NaN               | NaN               |
| 3        | 211951.589 | 24669.181         | 10.426            |
| 4        | 194366.277 | 17585.312         | 8.297             |
| 5        | 183343.423 | 11022.854         | 5.671             |
| 6        | 173018.552 | 10324.871         | 5.631             |
| 7        | 163687.848 | 9330.704          | 5.393             |
| 8        | 155471.634 | 8216.214          | 5.019             |
| 9        | 150575.355 | 4896.279          | 3.149             |
| 10       | 146418.241 | 4157.114          | 2.761             |



## Silhouette Analysis

```
[7]: silhouette_results = []  
  
cluster_labels_by_k = {}  
  
for k in candidate_k_values:
```

```

model = KMeans(
    n_clusters=k,
    n_init=10,
    random_state=42
)
labels = model.fit_predict(X_scaled_full)
cluster_labels_by_k[k] = labels

silhouette_results.append({
    "Clusters": k,
    "Silhouette": silhouette_score(
        X_scaled_full,
        labels,
        sample_size=5000,
        random_state=42
    ),
    "Davies-Bouldin": davies_bouldin_score(X_scaled_full, labels),
    "Calinski-Harabasz": calinski_harabasz_score(X_scaled_full, labels),
    "Smallest Cluster Share": (
        pd.Series(labels).value_counts(normalize=True).min()
    )
})

silhouette_df = pd.DataFrame(silhouette_results)

print("Internal Validation Criteria Across k")
print(silhouette_df.round(4).to_string(index=False))

best_silhouette_row = silhouette_df.loc[
    silhouette_df["Silhouette"].idxmax()
]
best_davies_row = silhouette_df.loc[
    silhouette_df["Davies-Bouldin"].idxmin()
]

selected_k = int(best_silhouette_row["Clusters"])

print(
    f"\nHighest silhouette at k={selected_k} "
    f"({best_silhouette_row['Silhouette']:.4f})"
)
print(
    "Lowest Davies-Bouldin at k="
    f"{int(best_davies_row['Clusters'])} "
    f"({best_davies_row['Davies-Bouldin']:.4f})"
)

```

```

if int(best_davies_row["Clusters"]) == selected_k:
    print(
        "Silhouette and Davies-Bouldin agree on the same k, "
        "which is reassuring given that they are computed differently."
    )
else:
    print(
        "The two criteria disagree, which is common on "
        "continuous data where cluster boundaries are gradual rather than "
        "sharp. Section 8 profiles the candidates so the choice can rest on "
        "business interpretability as well as internal geometry."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(
    silhouette_df["Clusters"],
    silhouette_df["Silhouette"],
    marker="o",
    linewidth=2
)
axes[0].axvline(
    selected_k,
    linestyle="--",
    label=f"Selected k = {selected_k}"
)
axes[0].set_xlabel("Number of Clusters (k)")
axes[0].set_ylabel("Silhouette Score")
axes[0].set_title("Silhouette Score Across k (higher is better)")
axes[0].legend()
axes[0].grid(alpha=0.25)

axes[1].plot(
    silhouette_df["Clusters"],
    silhouette_df["Davies-Bouldin"],
    marker="s",
    linewidth=2
)
axes[1].set_xlabel("Number of Clusters (k)")
axes[1].set_ylabel("Davies-Bouldin Index")
axes[1].set_title("Davies-Bouldin Across k (lower is better)")
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()

```

Internal Validation Criteria Across k

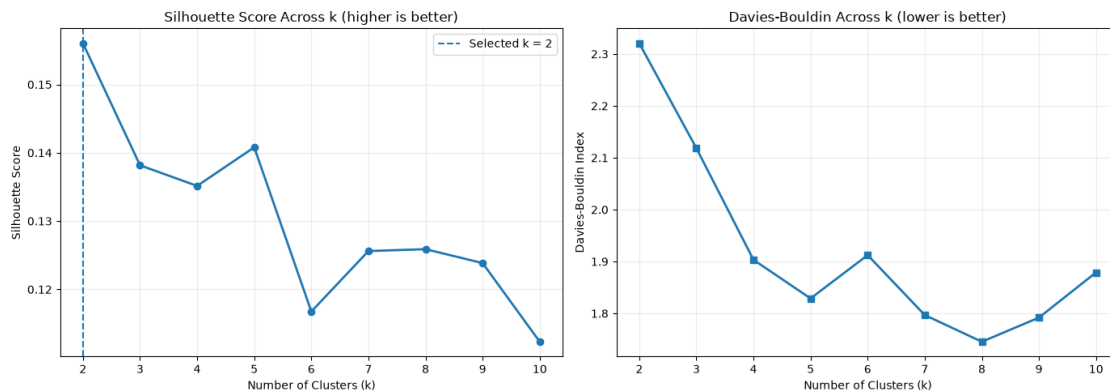
| Clusters | Silhouette | Davies-Bouldin | Calinski-Harabasz | Smallest Cluster Share |
|----------|------------|----------------|-------------------|------------------------|
|----------|------------|----------------|-------------------|------------------------|

|    |        |        |           |        |
|----|--------|--------|-----------|--------|
| 2  | 0.1560 | 2.3205 | 4982.6078 | 0.3415 |
| 3  | 0.1382 | 2.1184 | 4667.0599 | 0.2492 |
| 4  | 0.1351 | 1.9033 | 4370.0550 | 0.1207 |
| 5  | 0.1408 | 1.8288 | 3961.5275 | 0.0562 |
| 6  | 0.1168 | 1.9123 | 3744.9664 | 0.0443 |
| 7  | 0.1256 | 1.7968 | 3606.4344 | 0.0324 |
| 8  | 0.1259 | 1.7454 | 3499.1069 | 0.0027 |
| 9  | 0.1238 | 1.7919 | 3292.8839 | 0.0020 |
| 10 | 0.1123 | 1.8790 | 3112.2179 | 0.0020 |

Highest silhouette at k=2 (0.1560)

Lowest Davies-Bouldin at k=8 (1.7454)

The two criteria disagree, which is common on continuous data where cluster boundaries are gradual rather than sharp. Section 8 profiles the candidates so the choice can rest on business interpretability as well as internal geometry.



## Per-cluster silhouette diagrams

```
[8]: diagram_k_values = [
    max(2, selected_k - 1),
    selected_k,
    selected_k + 1
]

diagram_k_values = sorted(set(diagram_k_values))

diagram_sample_size = 5000

diagram_rng = np.random.default_rng(42)
diagram_indices = diagram_rng.choice(
    X_scaled_full.shape[0],
    size=min(diagram_sample_size, X_scaled_full.shape[0]),
    replace=False
)
```

```

X_diagram = X_scaled_full[diagram_indices]

fig, axes = plt.subplots(
    1,
    len(diagram_k_values),
    figsize=(6 * len(diagram_k_values), 6)
)

if len(diagram_k_values) == 1:
    axes = [axes]

for axis, k in zip(axes, diagram_k_values):
    labels_sample = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    ).fit_predict(X_diagram)

    average_silhouette = silhouette_score(X_diagram, labels_sample)
    sample_silhouettes = silhouette_samples(X_diagram, labels_sample)

    lower_bound = 10

    for cluster in range(k):
        cluster_values = np.sort(
            sample_silhouettes[labels_sample == cluster]
        )
        cluster_size = cluster_values.shape[0]
        upper_bound = lower_bound + cluster_size

        colour = cm.nipy_spectral(float(cluster) / k)

        axis.fill_betweenx(
            np.arange(lower_bound, upper_bound),
            0,
            cluster_values,
            facecolor=colour,
            edgecolor=colour,
            alpha=0.75
        )

        axis.text(
            -0.05,
            lower_bound + 0.5 * cluster_size,
            str(cluster)
        )

```

```

        lower_bound = upper_bound + 10

    axis.axvline(
        average_silhouette,
        color="red",
        linestyle="--",
        label=f"Mean = {average_silhouette:.3f}"
    )
    axis.set_xlabel("Silhouette Coefficient")
    axis.set_ylabel("Borrowers, Grouped by Cluster")
    axis.set_title(f"Silhouette Diagram at k={k}")
    axis.set_yticks([])
    axis.legend(loc="lower right")

plt.tight_layout()
plt.show()

negative_share_results = []

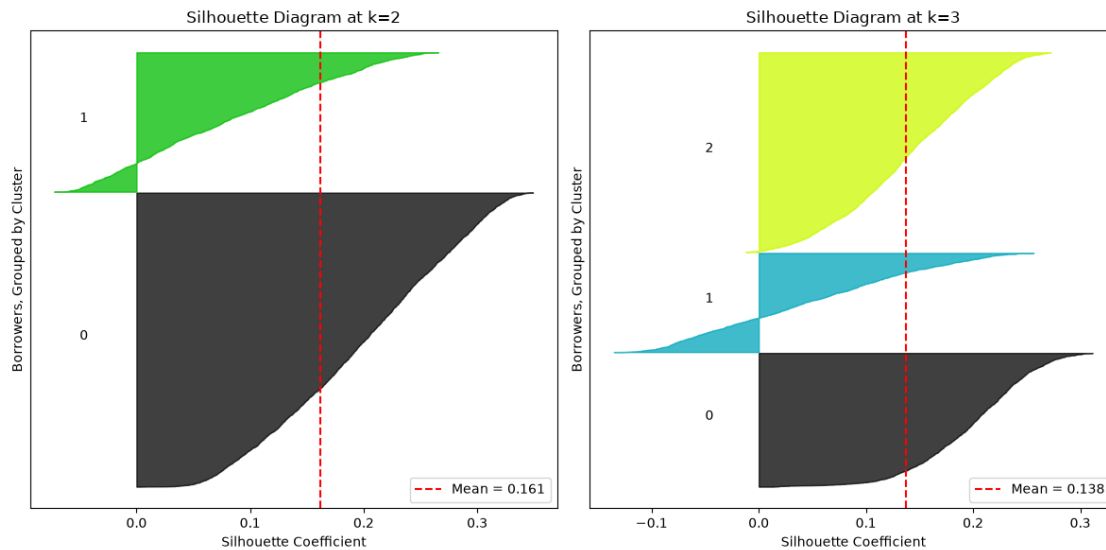
for k in diagram_k_values:
    labels_sample = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    ).fit_predict(X_diagram)

    sample_silhouettes = silhouette_samples(X_diagram, labels_sample)

    negative_share_results.append({
        "Clusters": k,
        "Mean Silhouette": sample_silhouettes.mean(),
        "Share Below Zero": (sample_silhouettes < 0).mean(),
        "Weakest Cluster Mean": min(
            sample_silhouettes[labels_sample == cluster].mean()
            for cluster in range(k)
        )
    })

print("Silhouette Distribution Detail")
print(pd.DataFrame(negative_share_results).round(4).to_string(index=False))

```



#### Silhouette Distribution Detail

| Clusters | Mean Silhouette | Share Below Zero | Weakest Cluster Mean |
|----------|-----------------|------------------|----------------------|
| 2        | 0.1615          | 0.0662           | 0.0827               |
| 3        | 0.1378          | 0.0818           | 0.0483               |

#### Feature Space: Do the One-Hot Indicators Belong in the Distance?

```
[9]: feature_space_results = []

feature_space_labels = {}

for label, matrix in [
    ("Numeric only", X_numeric_only),
    ("Numeric plus one-hot", X_scaled_full)
]:
    labels = KMeans(
        n_clusters=selected_k,
        n_init=10,
        random_state=42
    ).fit_predict(matrix)

    feature_space_labels[label] = labels

    debt_burden_by_cluster = pd.Series(
        df_model["loan_percent_income"].values
    ).groupby(labels).mean()

    default_rate_by_cluster = pd.Series(
        y_holdout.values
```

```

).groupby(labels).mean()

feature_space_results.append({
    "Feature Space": label,
    "Dimensions": matrix.shape[1],
    "Silhouette": silhouette_score(
        matrix,
        labels,
        sample_size=5000,
        random_state=42
    ),
    "Davies-Bouldin": davies_bouldin_score(matrix, labels),
    "Debt Burden Spread": (
        debt_burden_by_cluster.max()
        - debt_burden_by_cluster.min()
    ),
    "Default Rate Spread": (
        default_rate_by_cluster.max()
        - default_rate_by_cluster.min()
    )
})

feature_space_df = pd.DataFrame(feature_space_results)

print(f"Feature Space Comparison at k={selected_k}")
print(feature_space_df.round(4).to_string(index=False))

space_agreement = adjusted_rand_score(
    feature_space_labels["Numeric only"],
    feature_space_labels["Numeric plus one-hot"]
)

print(
    f"\nAdjusted Rand index between the two partitions: {space_agreement:.4f}"
)

if space_agreement > 0.7:
    print(
        " The two representations recover substantially the same "
        "segments, so the categorical indicators are not driving the "
        "partition and the choice between spaces is low-stakes."
    )
else:
    print(
        "The two representations produce materially different "
        "segments. The categorical indicators are exerting real influence on "
        "the distance metric, so the choice of feature space is itself a "

```

```

        "substantive modelling decision rather than a technicality."
    )

    if (
        feature_space_df.loc[1, "Default Rate Spread"]
        >= feature_space_df.loc[0, "Default Rate Spread"]
    ):
        selected_matrix = X_scaled_full
        selected_space_label = "numeric plus one-hot"
    else:
        selected_matrix = X_numeric_only
        selected_space_label = "numeric only"

    print(
        "\nFeature space carried forward (chosen on risk separation, "
        f"not on internal geometry): {selected_space_label}"
    )

```

Feature Space Comparison at k=2

|                     | Feature Space        | Dimensions | Silhouette | Davies-Bouldin | Debt Burden Spread |
|---------------------|----------------------|------------|------------|----------------|--------------------|
| Default Rate Spread |                      |            |            |                |                    |
|                     | Numeric only         | 6          | 0.2215     | 1.8216         | 0.1591             |
| 0.2241              |                      |            |            |                |                    |
|                     | Numeric plus one-hot | 25         | 0.1560     | 2.3205         | 0.1544             |
| 0.2410              |                      |            |            |                |                    |

Adjusted Rand index between the two partitions: 0.8602

The two representations recover substantially the same segments, so the categorical indicators are not driving the partition and the choice between spaces is low-stakes.

Feature space carried forward (chosen on risk separation, not on internal geometry): numeric plus one-hot

## Distance Metrics and K-Means

**Spherical K-Means** projects every borrower onto the unit sphere by L2-normalising their feature vector, then runs ordinary K-Means. On unit-length vectors, squared Euclidean distance is a monotone function of cosine distance, so this genuinely clusters by cosine similarity while keeping the standard algorithm valid. Substantively it discards magnitude and clusters on *profile shape*: two borrowers with the same balance of income to debt burden to rate land together even at very different absolute scales.

**K-Medoids** replaces the mean with the member minimising total distance to its cluster, which is a valid update for any metric. This admits Manhattan distance, which Week 8 found more robust to the outliers in the income distribution. Because it requires a full pairwise distance matrix it is run on a stratified subsample.

```

[10]: def k_medoids(distance_matrix, n_clusters, random_state=42, max_iterations=60):
    rng = np.random.default_rng(random_state)
    n_points = distance_matrix.shape[0]

    medoid_indices = rng.choice(
        n_points,
        size=n_clusters,
        replace=False
    )

    for _ in range(max_iterations):
        assignments = np.argmin(
            distance_matrix[:, medoid_indices],
            axis=1
        )

        updated_medoids = medoid_indices.copy()

        for cluster in range(n_clusters):
            members = np.where(assignments == cluster)[0]

            if members.size == 0:
                continue

            within_cluster_totals = distance_matrix[
                np.ix_(members, members)
            ].sum(axis=1)

            updated_medoids[cluster] = members[
                np.argmin(within_cluster_totals)
            ]

            if np.array_equal(
                np.sort(updated_medoids),
                np.sort(medoid_indices)
            ):
                medoid_indices = updated_medoids
                break

        medoid_indices = updated_medoids

    assignments = np.argmin(
        distance_matrix[:, medoid_indices],
        axis=1
    )

    return medoid_indices, assignments

```

```

metric_sample_size = 4000

metric_rng = np.random.default_rng(42)
metric_indices = metric_rng.choice(
    selected_matrix.shape[0],
    size=min(metric_sample_size, selected_matrix.shape[0]),
    replace=False
)

X_metric = selected_matrix[metric_indices]
y_metric = y_holdout.values[metric_indices]

euclidean_labels = KMeans(
    n_clusters=selected_k,
    n_init=10,
    random_state=42
).fit_predict(X_metric)

X_normalised = normalize(X_metric)

spherical_labels = KMeans(
    n_clusters=selected_k,
    n_init=10,
    random_state=42
).fit_predict(X_normalised)

manhattan_distances = pairwise_distances(
    X_metric,
    metric="manhattan"
)

_, manhattan_labels = k_medoids(
    distance_matrix=manhattan_distances,
    n_clusters=selected_k,
    random_state=42
)

metric_comparison = []

for label, labels, matrix, metric_name in [
    ("K-Means (Euclidean)", euclidean_labels, X_metric, "euclidean"),
    ("Spherical K-Means (cosine)", spherical_labels, X_normalised, "cosine"),
    ("K-Medoids (Manhattan)", manhattan_labels, X_metric, "manhattan")
]:
    default_rate_by_cluster = pd.Series(y_metric).groupby(labels).mean()

```

```

metric_comparison.append({
    "Method": label,
    "Silhouette (own metric)": silhouette_score(
        matrix,
        labels,
        metric=metric_name
    ),
    "Clusters Found": len(np.unique(labels)),
    "Smallest Cluster Share": (
        pd.Series(labels).value_counts(normalize=True).min()
    ),
    "Default Rate Spread": (
        default_rate_by_cluster.max()
        - default_rate_by_cluster.min()
    )
})

metric_comparison_df = pd.DataFrame(metric_comparison)

print(f"Distance Geometry Comparison at k={selected_k}")
print(f"Subsample: {len(metric_indices):,} borrowers")
print(metric_comparison_df.round(4).to_string(index=False))

print("\nPartition Agreement (Adjusted Rand Index)")
print(
    "Euclidean vs spherical :",
    f"{adjusted_rand_score(euclidean_labels, spherical_labels):.4f}"
)
print(
    "Euclidean vs Manhattan :",
    f"{adjusted_rand_score(euclidean_labels, manhattan_labels):.4f}"
)
print(
    "Spherical vs Manhattan :",
    f"{adjusted_rand_score(spherical_labels, manhattan_labels):.4f}"
)

print(
    "\n Silhouette values in the table are each computed under "
    "their own metric and are therefore not directly comparable across rows. "
    "The Rand indices and the default-rate spread are the comparable "
    "quantities: they show whether the three geometries are describing the "
    "same borrower population differently or finding genuinely different "
    "structure."
)

```

Distance Geometry Comparison at k=2

Subsample: 4,000 borrowers

|               | Method                     | Silhouette (own metric) | Clusters Found | Smallest |
|---------------|----------------------------|-------------------------|----------------|----------|
| Cluster Share | Default Rate               | Spread                  |                |          |
|               | K-Means (Euclidean)        | 0.1684                  | 2              |          |
| 0.3135        | 0.2275                     |                         |                |          |
|               | Spherical K-Means (cosine) | 0.2365                  | 2              |          |
| 0.4868        | 0.2307                     |                         |                |          |
|               | K-Medoids (Manhattan)      | 0.1147                  | 2              |          |
| 0.4440        | 0.0267                     |                         |                |          |

Partition Agreement (Adjusted Rand Index)

Euclidean vs spherical : 0.2259

Euclidean vs Manhattan : 0.1257

Spherical vs Manhattan : 0.1956

Silhouette values in the table are each computed under their own metric and are therefore not directly comparable across rows. The Rand indices and the default-rate spread are the comparable quantities: they show whether the three geometries are describing the same borrower population differently or finding genuinely different structure.

### Cluster Profiles in Business Terms

```
[11]: final_kmeans = KMeans(
        n_clusters=selected_k,
        n_init=25,
        random_state=42
    )

    final_labels = final_kmeans.fit_predict(selected_matrix)

    profile_frame = df_model.copy()
    profile_frame["cluster"] = final_labels

    cluster_profile = profile_frame.groupby("cluster").agg(
        Borrowers=("loan_status", "size"),
        Median_Income=("person_income", "median"),
        Median_Loan=("loan_amnt", "median"),
        Mean_Debt_Burden=("loan_percent_income", "mean"),
        Mean_Interest_Rate=("loan_int_rate", "mean"),
        Median_Age=("person_age", "median"),
        Mean_Employment_Years=("person_emp_length", "mean"),
        Default_Rate=("loan_status", "mean")
    ).round(4)

    cluster_profile["Population Share"] = (
        cluster_profile["Borrowers"]
        / cluster_profile["Borrowers"].sum()
```

```

).round(4)

print(f"Cluster Profiles at k={selected_k}")
print(cluster_profile.to_string())

overall_default_rate = y_holdout.mean()

cluster_profile_sorted = cluster_profile.sort_values("Default_Rate")

default_spread = (
    cluster_profile["Default_Rate"].max()
    - cluster_profile["Default_Rate"].min()
)

print(f"\nPopulation default rate: {overall_default_rate:.4f}")
print(f"Lowest-risk cluster default rate: {cluster_profile['Default_Rate'].
    ↪min():.4f}")
print(f"Highest-risk cluster default rate: {cluster_profile['Default_Rate'].
    ↪max():.4f}")
print(f"Spread across clusters: {default_spread:.4f}")
print(
    "Risk ratio, highest to lowest cluster:",
    f"{cluster_profile['Default_Rate'].max() / ↪
    ↪max(cluster_profile['Default_Rate'].min(), 1e-9):.2f}x"
)

if default_spread > 0.10:
    print(
        "\n The segments differ substantially in realised default "
        "rate despite the label never entering the clustering. The geometry "
        "of borrower attributes carries genuine risk information, which is "
        "the premise the supervised models of Weeks 4 through 9 relied on."
    )
else:
    print(
        "\n Default rates are similar across segments. K-Means "
        "has partitioned this population along dimensions largely orthogonal "
        "to credit risk, which is an honest negative result and a real "
        "limitation of unsupervised segmentation on this dataset."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(
    cluster_profile_sorted.index.astype(str),
    cluster_profile_sorted["Default_Rate"]
)

```

```

axes[0].axhline(
    overall_default_rate,
    linestyle="--",
    color="red",
    label=f"Population rate = {overall_default_rate:.3f}"
)
axes[0].set_xlabel("Cluster")
axes[0].set_ylabel("Realised Default Rate")
axes[0].set_title("Default Rate by Cluster (label withheld during fitting)")
axes[0].legend()
axes[0].grid(axis="y", alpha=0.25)

axes[1].scatter(
    cluster_profile["Median_Income"],
    cluster_profile["Mean_Debt_Burden"],
    s=cluster_profile["Borrowers"] / 20,
    c=cluster_profile["Default_Rate"],
    cmap="coolwarm"
)
for cluster_id, row in cluster_profile.iterrows():
    axes[1].annotate(
        str(cluster_id),
        (row["Median_Income"], row["Mean_Debt_Burden"])
    )
axes[1].set_xlabel("Median Income")
axes[1].set_ylabel("Mean Debt Burden (loan as share of income)")
axes[1].set_title("Segment Positions, Sized by Population, Coloured by Risk")
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()

```

Cluster Profiles at k=2

|                    | Borrowers  | Median_Income         | Median_Loan  | Mean_Debt_Burden |       |        |
|--------------------|------------|-----------------------|--------------|------------------|-------|--------|
| Mean_Interest_Rate | Median_Age | Mean_Employment_Years | Default_Rate | Population       | Share |        |
| cluster            |            |                       |              |                  |       |        |
| 0                  | 11064      | 57196.0               | 15000.0      | 0.2719           |       |        |
| 12.5280            | 26.0       |                       | 4.9371       | 0.3774           |       | 0.3414 |
| 1                  | 21345      | 55000.0               | 6000.0       | 0.1176           |       |        |
| 10.2384            | 26.0       |                       | 4.6704       | 0.1364           |       | 0.6586 |

Population default rate: 0.2187

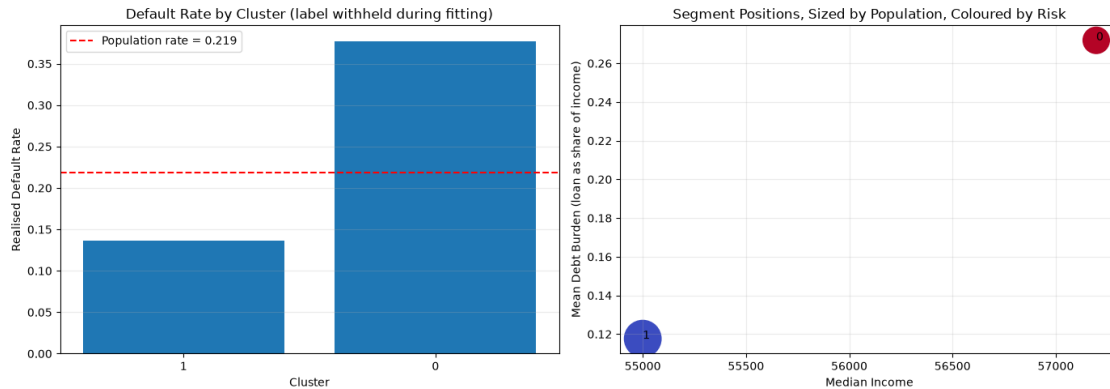
Lowest-risk cluster default rate: 0.1364

Highest-risk cluster default rate: 0.3774

Spread across clusters: 0.2410

Risk ratio, highest to lowest cluster: 2.77x

The segments differ substantially in realised default rate despite the label never entering the clustering. The geometry of borrower attributes carries genuine risk information, which is the premise the supervised models of Weeks 4 through 9 relied on.



## Visualising the Clusters in Two Dimensions

```
[12]: pca = PCA(n_components=2, random_state=42)
X_projected = pca.fit_transform(selected_matrix)

explained = pca.explained_variance_ratio_

print(f"Variance explained by component 1: {explained[0]:.4f}")
print(f"Variance explained by component 2: {explained[1]:.4f}")
print(f"Total variance captured in this projection: {explained.sum():.4f}")

plot_rng = np.random.default_rng(42)
plot_indices = plot_rng.choice(
    X_projected.shape[0],
    size=min(6000, X_projected.shape[0]),
    replace=False
)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

scatter_clusters = axes[0].scatter(
    X_projected[plot_indices, 0],
    X_projected[plot_indices, 1],
    c=final_labels[plot_indices],
    cmap="tab10",
    s=6,
    alpha=0.5
)
axes[0].set_xlabel(f"Component 1 ({explained[0]:.1%} of variance)")
```

```

axes[0].set_ylabel(f"Component 2 ({explained[1]:.1%} of variance)")
axes[0].set_title(f"K-Means Segments at k={selected_k}")

scatter_defaults = axes[1].scatter(
    X_projected[plot_indices, 0],
    X_projected[plot_indices, 1],
    c=y_holdout.values[plot_indices],
    cmap="coolwarm",
    s=6,
    alpha=0.5
)
axes[1].set_xlabel(f"Component 1 ({explained[0]:.1%} of variance)")
axes[1].set_ylabel(f"Component 2 ({explained[1]:.1%} of variance)")
axes[1].set_title("The Same Projection Coloured by Withheld Default Status")

plt.tight_layout()
plt.show()

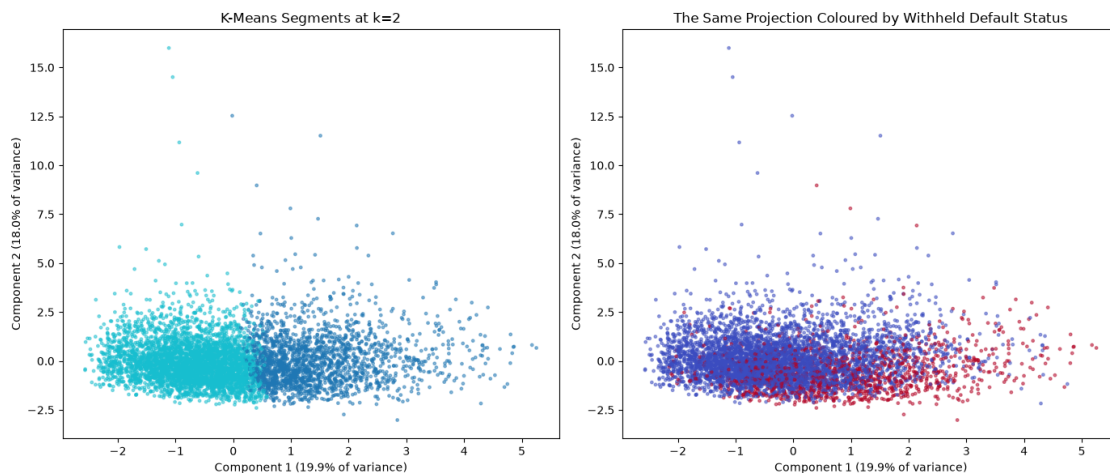
print(
    "\n Comparing the two panels shows whether the cluster "
    "boundaries K-Means drew align with where defaults actually concentrate. "
    "Visible correspondence supports the default-rate spread measured in "
    "Section 8; visible mismatch means the segments are capturing structure "
    "that is real but not risk-related."
)

```

Variance explained by component 1: 0.1995

Variance explained by component 2: 0.1802

Total variance captured in this projection: 0.3797



Comparing the two panels shows whether the cluster boundaries K-Means drew

align with where defaults actually concentrate. Visible correspondence supports the default-rate spread measured in Section 8; visible mismatch means the segments are capturing structure that is real but not risk-related.

### Stability Under Different Scaling and Feature Choices

Four perturbations are applied, each defensible and each one an analyst could plausibly have chosen instead:

- **RobustScaler** centres on the median and scales by the interquartile range, so the extreme incomes in this dataset pull the transformation far less than they do under `StandardScaler`.
- **MinMaxScaler** compresses every feature into a fixed zero-to-one range, which changes the relative weight of the one-hot indicators against the continuous attributes.
- **Dropping `loan_grade`** removes the lender-assigned artifact identified in Week 3, leaving only attributes intrinsic to the borrower.
- **A different random seed** changes centroid initialisation, testing whether the algorithm is even converging to the same solution on identical data.

```
[13]: robust_preprocessor = ColumnTransformer(
    transformers=[
        ("num", RobustScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

minmax_preprocessor = ColumnTransformer(
    transformers=[
        ("num", MinMaxScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

reduced_categoricals = [
    column for column in categorical_features
    if column != "loan_grade"
]

no_grade_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
```

```

        "cat",
        OneHotEncoder(handle_unknown="ignore", sparse_output=False),
        reduced_categoricals
    )
]
)

stability_variants = [
    (
        "Baseline (StandardScaler, seed 42)",
        selected_matrix,
        42
    ),
    (
        "RobustScaler",
        np.asarray(robust_preprocessor.fit_transform(X), dtype=np.float64),
        42
    ),
    (
        "MinMaxScaler",
        np.asarray(minmax_preprocessor.fit_transform(X), dtype=np.float64),
        42
    ),
    (
        "loan_grade removed",
        np.asarray(no_grade_preprocessor.fit_transform(X), dtype=np.float64),
        42
    ),
    (
        "Different seed (seed 7)",
        selected_matrix,
        7
    )
]

stability_results = []

baseline_labels = None

for label, matrix, seed in stability_variants:
    labels = KMeans(
        n_clusters=selected_k,
        n_init=10,
        random_state=seed
    ).fit_predict(matrix)

    if baseline_labels is None:

```

```

baseline_labels = labels

default_rate_by_cluster = pd.Series(
    y_holdout.values
).groupby(labels).mean()

stability_results.append({
    "Variant": label,
    "Agreement With Baseline": adjusted_rand_score(
        baseline_labels,
        labels
    ),
    "Silhouette": silhouette_score(
        matrix,
        labels,
        sample_size=5000,
        random_state=42
    ),
    "Default Rate Spread": (
        default_rate_by_cluster.max()
        - default_rate_by_cluster.min()
    ),
    "Smallest Cluster Share": (
        pd.Series(labels).value_counts(normalize=True).min()
    )
})

stability_df = pd.DataFrame(stability_results)

print(f"Cluster Stability at k={selected_k}")
print(stability_df.round(4).to_string(index=False))

perturbation_agreements = stability_df.loc[:, "Agreement With Baseline"]
weakest_agreement = perturbation_agreements.min()
weakest_variant = stability_df.loc[
    perturbation_agreements.idxmin(),
    "Variant"
]

print(f"\nWeakest agreement: {weakest_variant} at {weakest_agreement:.4f}")

if weakest_agreement > 0.6:
    print(
        " The segmentation survives every perturbation tested. "
        "The clusters reflect structure in the borrower population rather "
        "than an accident of one preprocessing choice, which is the "
        "precondition for reporting them to a decision-maker."
    )

```

```

    )
elif weakest_agreement > 0.3:
    print(
        "The partition is moderately stable. Its broad shape "
        "survives, but individual borrowers near cluster boundaries move "
        "between segments depending on preprocessing, so segment membership "
        "should not be treated as a fixed property of a borrower."
    )
else:
    print(
        " The partition is not stable under reasonable "
        "preprocessing changes. This is the most important finding in the "
        "notebook and it argues against operationalising these specific "
        "segments, whatever their silhouette score suggests."
    )

plt.figure(figsize=(10, 5))
plt.barh(
    stability_df["Variant"],
    stability_df["Agreement With Baseline"]
)
for index, value in enumerate(stability_df["Agreement With Baseline"]):
    plt.text(
        value + 0.01,
        index,
        f"{value:.3f}",
        va="center"
    )
plt.xlabel("Adjusted Rand Index Against Baseline Partition")
plt.title("Does the Segmentation Survive Different Preprocessing Choices?")
plt.xlim(0, 1.1)
plt.tight_layout()
plt.show()

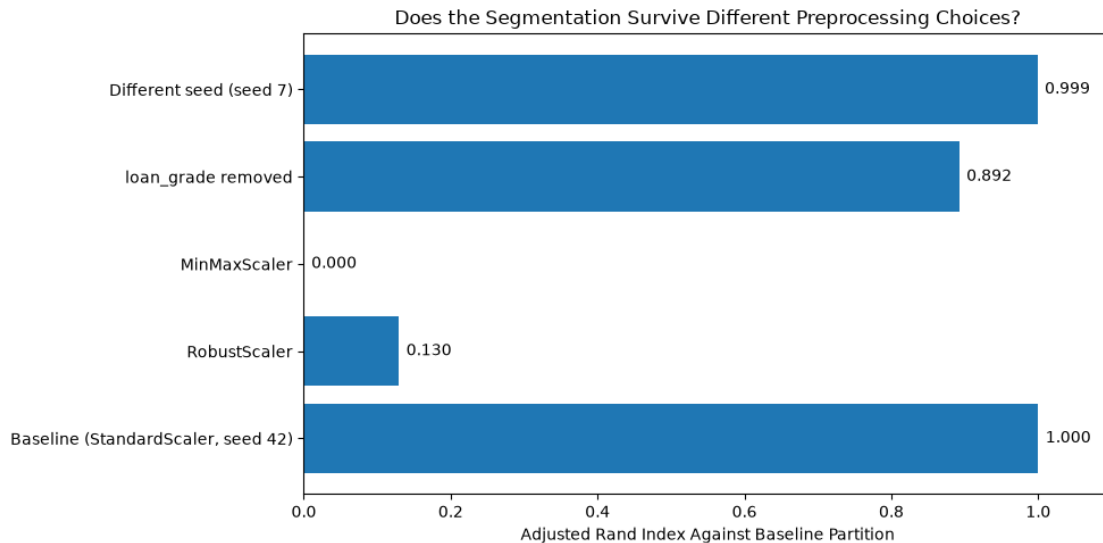
```

Cluster Stability at k=2

|                                    | Variant                 | Agreement With Baseline | Silhouette | Default |
|------------------------------------|-------------------------|-------------------------|------------|---------|
| Rate Spread                        |                         |                         |            |         |
| Smallest Cluster Share             |                         |                         |            |         |
| Baseline (StandardScaler, seed 42) |                         | 1.0000                  | 0.1560     |         |
| 0.2410                             | 0.3415                  |                         |            |         |
|                                    | RobustScaler            | 0.1295                  | 0.1863     |         |
| 0.0798                             | 0.2571                  |                         |            |         |
|                                    | MinMaxScaler            | 0.0003                  | 0.1794     |         |
| 0.1978                             | 0.4943                  |                         |            |         |
|                                    | loan_grade removed      | 0.8923                  | 0.1675     |         |
| 0.2348                             | 0.3345                  |                         |            |         |
|                                    | Different seed (seed 7) | 0.9989                  | 0.1561     |         |
| 0.2410                             | 0.3415                  |                         |            |         |

Weakest agreement: MinMaxScaler at 0.0003

The partition is not stable under reasonable preprocessing changes. This is the most important finding in the notebook and it argues against operationalising these specific segments, whatever their silhouette score suggests.



## Does Clustering Improve the Supervised Model?

Two versions are tested against the Week 9 tuned gradient boosting benchmark:

1. **Cluster as a feature** - the segment label is appended to the feature matrix as one-hot indicators and a single model is fit as before.
2. **Per-cluster specialists** - a separate gradient boosting model is fit within each segment, and test borrowers are scored by whichever specialist owns their segment.

The leakage discipline matters here. K-Means is fit on the training features only, then used to assign segments to test borrowers, exactly as it would be in production where tomorrow's applicants were not available when the segmentation was built.

```
[14]: X_supervised_train, X_supervised_test, y_supervised_train, y_supervised_test = (
    train_test_split(
        X,
        y_holdout,
        test_size=0.20,
        random_state=42,
        stratify=y_holdout
    )
)

supervised_encoder = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
```

```

        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

X_sup_train_enc = np.asarray(
    supervised_encoder.fit_transform(X_supervised_train),
    dtype=np.float32
)
X_sup_test_enc = np.asarray(
    supervised_encoder.transform(X_supervised_test),
    dtype=np.float32
)

clustering_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

X_cluster_train = clustering_preprocessor.fit_transform(X_supervised_train)
X_cluster_test = clustering_preprocessor.transform(X_supervised_test)

segmenter = KMeans(
    n_clusters=selected_k,
    n_init=25,
    random_state=42
)

train_segments = segmenter.fit_predict(X_cluster_train)
test_segments = segmenter.predict(X_cluster_test)

print(f"Segments assigned using k={selected_k}")
print("\nTraining segment sizes and default rates")
print(
    pd.DataFrame({
        "Segment": train_segments,
        "Default": np.asarray(y_supervised_train)
    }).groupby("Segment").agg(

```

```

        Borrowers=("Default", "size"),
        Default_Rate=("Default", "mean")
    ).round(4).to_string()
)

def fit_gradient_boosting(X_fit, y_fit):
    model = GradientBoostingClassifier(
        subsample=0.85,
        n_estimators=300,
        min_samples_leaf=3,
        max_features=None,
        max_depth=4,
        learning_rate=0.10,
        random_state=42
    )

    weights = compute_sample_weight(
        class_weight="balanced",
        y=y_fit
    )

    model.fit(X_fit, y_fit, sample_weight=weights)

    return model

baseline_model = fit_gradient_boosting(
    X_sup_train_enc,
    y_supervised_train
)

baseline_probabilities = baseline_model.predict_proba(
    X_sup_test_enc
)[: , 1]

segment_indicator_train = np.zeros(
    (len(train_segments), selected_k),
    dtype=np.float32
)
segment_indicator_train[
    np.arange(len(train_segments)),
    train_segments
] = 1.0

segment_indicator_test = np.zeros(
    (len(test_segments), selected_k),

```

```

        dtype=np.float32
    )
    segment_indicator_test[
        np.arange(len(test_segments)),
        test_segments
    ] = 1.0

    X_augmented_train = np.hstack(
        [X_sup_train_enc, segment_indicator_train]
    )
    X_augmented_test = np.hstack(
        [X_sup_test_enc, segment_indicator_test]
    )

    augmented_model = fit_gradient_boosting(
        X_augmented_train,
        y_supervised_train
    )

    augmented_probabilities = augmented_model.predict_proba(
        X_augmented_test
    )[:, 1]

    specialist_probabilities = np.zeros(len(y_supervised_test))

    y_supervised_train_array = np.asarray(y_supervised_train)

    minimum_segment_size = 500

    for segment in range(selected_k):
        train_mask = train_segments == segment
        test_mask = test_segments == segment

        if test_mask.sum() == 0:
            continue

        segment_y = y_supervised_train_array[train_mask]

        if train_mask.sum() < minimum_segment_size or len(np.unique(segment_y)) < 2:
            specialist_probabilities[test_mask] = baseline_probabilities[test_mask]
            print(
                f"Segment {segment}: too small or single-class "
                f"({train_mask.sum()} training borrowers), "
                "falling back to the pooled model."
            )
            continue

```

```

specialist = fit_gradient_boosting(
    X_sup_train_enc[train_mask],
    segment_y
)

specialist_probabilities[test_mask] = specialist.predict_proba(
    X_sup_test_enc[test_mask]
)[: , 1]

two_stage_comparison = pd.DataFrame([
    {
        "Approach": "Pooled model (Week 9 specification)",
        "Test ROC AUC": roc_auc_score(
            y_supervised_test,
            baseline_probabilities
        ),
        "Test Average Precision": average_precision_score(
            y_supervised_test,
            baseline_probabilities
        )
    },
    {
        "Approach": "Pooled model plus cluster feature",
        "Test ROC AUC": roc_auc_score(
            y_supervised_test,
            augmented_probabilities
        ),
        "Test Average Precision": average_precision_score(
            y_supervised_test,
            augmented_probabilities
        )
    },
    {
        "Approach": "Per-cluster specialist models",
        "Test ROC AUC": roc_auc_score(
            y_supervised_test,
            specialist_probabilities
        ),
        "Test Average Precision": average_precision_score(
            y_supervised_test,
            specialist_probabilities
        )
    }
])

print("\nTwo-Stage Architecture Comparison")
print(two_stage_comparison.round(4).to_string(index=False))

```

```

augmented_gain = (
    two_stage_comparison.loc[1, "Test ROC AUC"]
    - two_stage_comparison.loc[0, "Test ROC AUC"]
)

specialist_gain = (
    two_stage_comparison.loc[2, "Test ROC AUC"]
    - two_stage_comparison.loc[0, "Test ROC AUC"]
)

print(f"\nCluster feature changes AUC by {augmented_gain:+.4f}")
print(f"Per-cluster specialists change AUC by {specialist_gain:+.4f}")

best_gain = max(augmented_gain, specialist_gain)

if best_gain > 0.002:
    print(
        "\n Pre-segmenting borrowers improves risk ranking. The "
        "gain must still be weighed against the cost of explaining a "
        "two-stage pipeline to a regulator, since a lender must be able to "
        "justify a declined application."
    )
else:
    print(
        "\nPre-segmenting does not improve on the pooled model. "
        "This is informative rather than disappointing: gradient boosting "
        "already partitions the feature space internally through its splits, "
        "so an explicit K-Means partition supplies information the trees had "
        "already recovered. The simpler single-stage pipeline is preferred, "
        "and it is also the easier one to defend to a regulator."
    )

plt.figure(figsize=(10, 5))
auc_order = two_stage_comparison.sort_values("Test ROC AUC")
plt.barh(
    auc_order["Approach"],
    auc_order["Test ROC AUC"]
)
for index, value in enumerate(auc_order["Test ROC AUC"]):
    plt.text(
        value + 0.001,
        index,
        f"{value:.4f}",
        va="center"
    )
plt.xlabel("Test ROC AUC")

```

```
plt.title("Does Clustering Before Classifying Help?")
plt.tight_layout()
plt.show()
```

Segments assigned using k=2

Training segment sizes and default rates

|           | Borrowers | Default_Rate |
|-----------|-----------|--------------|
| Segment 0 | 17086     | 0.1349       |
| Segment 1 | 8841      | 0.3806       |

Two-Stage Architecture Comparison

|                                     | Approach | Test ROC AUC | Test Average Precision |
|-------------------------------------|----------|--------------|------------------------|
| Pooled model (Week 9 specification) |          | 0.9538       | 0.9113                 |
| Pooled model plus cluster feature   |          | 0.9534       | 0.9109                 |
| Per-cluster specialist models       |          | 0.9466       | 0.9039                 |

Cluster feature changes AUC by -0.0003

Per-cluster specialists change AUC by -0.0072

Pre-segmenting does not improve on the pooled model. This is informative rather than disappointing: gradient boosting already partitions the feature space internally through its splits, so an explicit K-Means partition supplies information the trees had already recovered. The simpler single-stage pipeline is preferred, and it is also the easier one to defend to a regulator.

